

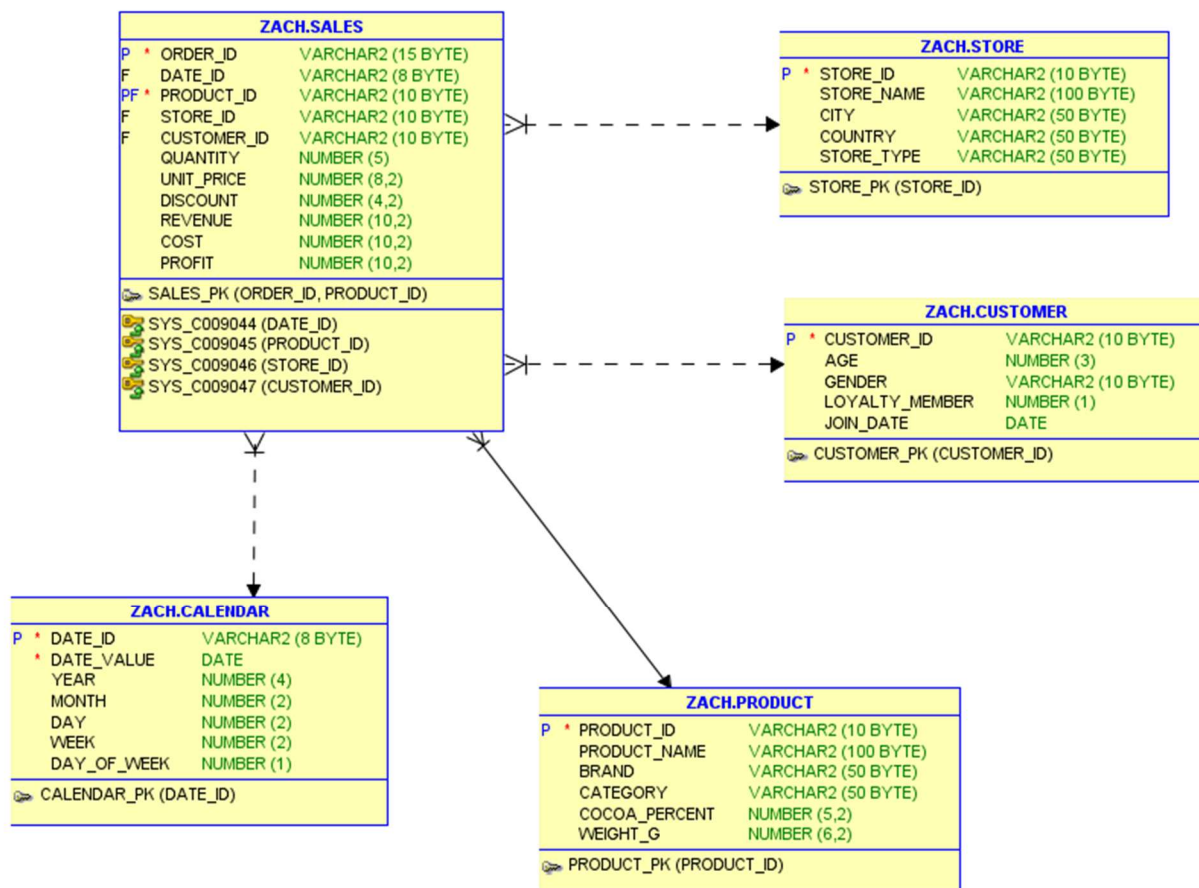
Project Title: Chocolate Sales Data Management and Analysis Using PL/SQL

This project focuses on building a chocolate sales database using PL/SQL to manage and analyze data more efficiently. The system organizes information across related tables such as calendar, customers, products, stores, and sales, allowing transactions to be tracked accurately while maintaining strong relationships between tables. The dataset, sourced from Kaggle, contains nearly 1 million sales records and required adjustments to meet third normal form (3NF) standards.

Along with the database design, PL/SQL is used to handle business logic and automate processes through tools like loops, cursors, procedures, functions, and exception handling. The database follows 3NF guidelines to reduce redundancy and maintain data integrity, creating a scalable and reliable structure for analyzing large-scale sales activity.

Link: <https://www.kaggle.com/datasets/saidaminsaidaxmadov/chocolate-sales/data>

ER Diagram



SQL Database

/*

--View tables in the DB

select * from tab;

--View Data in the table

select * from CALENDAR;

select * from CUSTOMER;

select * from PRODUCT;

select * from STORE;

select * from SALES;

-- Drop Tables

drop table sales;

drop table store;

drop table product;

drop table customer;

drop table calendar;

*/

-- 1. Create Calendar Table

```
CREATE TABLE calendar (  
    date_id    VARCHAR(8) PRIMARY KEY,  
    date_value DATE NOT NULL,  
    year       NUMBER(4),  
    month      NUMBER(2),  
    day        NUMBER(2),  
    week       NUMBER(2),  
    day_of_week NUMBER(1)  
);
```

-- 2. Create Customer Table

```
CREATE TABLE customer (  
    customer_id VARCHAR2(10) PRIMARY KEY,  
    age         NUMBER(3),  
    gender      VARCHAR2(10),  
    loyalty_member NUMBER(1) CHECK (loyalty_member IN (0,1)),  
    join_date   DATE  
);
```

-- 3. Create Product Table

```
CREATE TABLE product (  
    product_id  VARCHAR2(10) PRIMARY KEY,  
    product_name VARCHAR2(100),  
    brand       VARCHAR2(50),  
    category    VARCHAR2(50),  
    cocoa_percent NUMBER(5,2),  
    weight_g    NUMBER(6,2)  
);
```

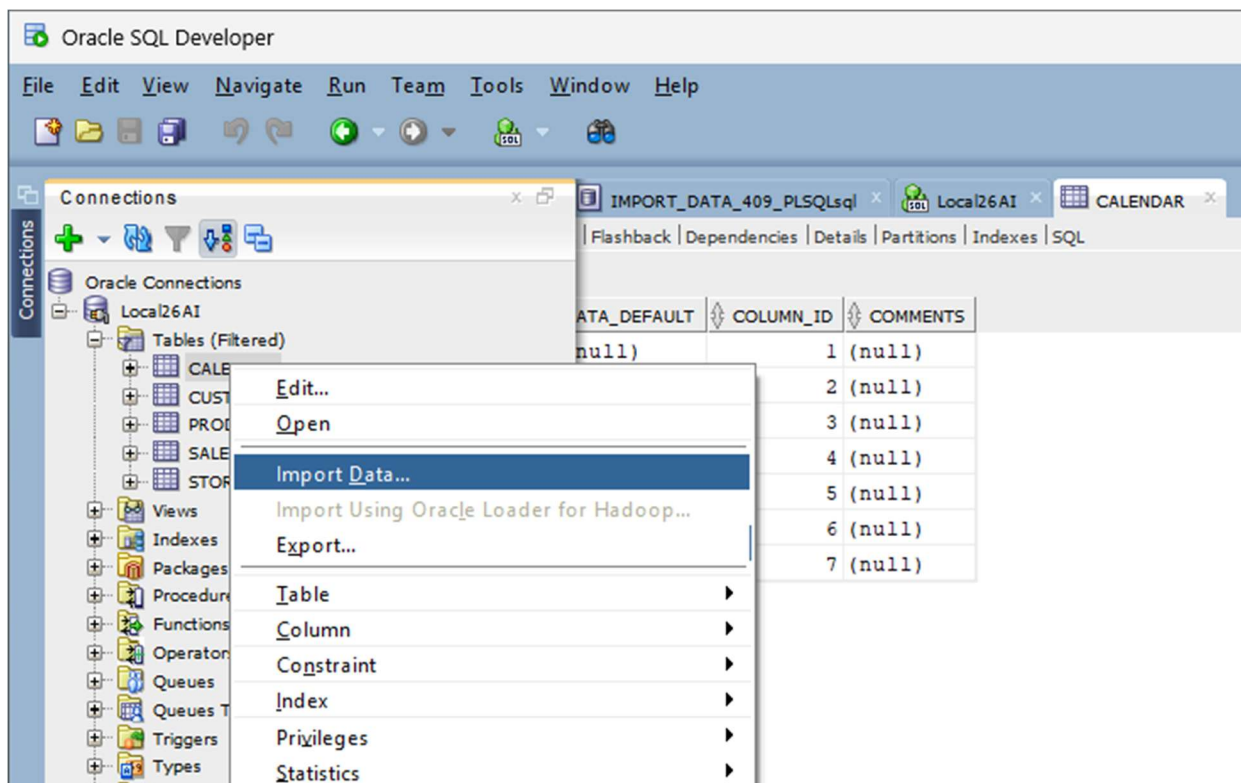
-- 4. Create Store Table

```
CREATE TABLE store (  
    store_id  VARCHAR2(10) PRIMARY KEY,  
    store_name VARCHAR2(100),  
    city      VARCHAR2(50),  
    country   VARCHAR2(50),  
    store_type VARCHAR2(50)  
);
```

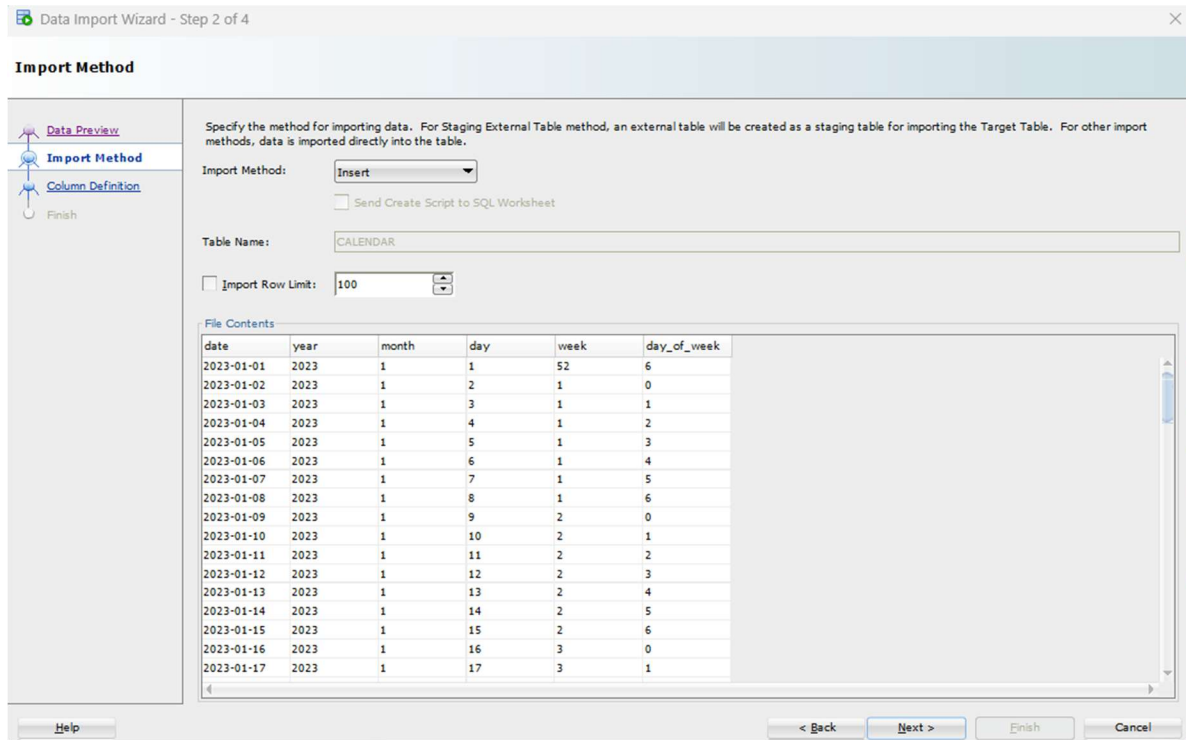
-- 5. Create Sales Table

```
CREATE TABLE sales (  
    order_id  VARCHAR2(15),  
    date_id   VARCHAR(8),  
    product_id VARCHAR2(10),  
    store_id  VARCHAR2(10),  
    customer_id VARCHAR2(10),  
    quantity  NUMBER(5),  
    unit_price NUMBER(8,2),  
    discount  NUMBER(4,2),  
    revenue   NUMBER(10,2),  
    cost      NUMBER(10,2),  
    profit    NUMBER(10,2),  
  
    PRIMARY KEY (order_id, product_id),  
  
    FOREIGN KEY (date_id) REFERENCES calendar(date_id),  
    FOREIGN KEY (product_id) REFERENCES product(product_id),  
    FOREIGN KEY (store_id) REFERENCES store(store_id),  
    FOREIGN KEY (customer_id) REFERENCES customer(customer_id)  
);
```

(This kaggle dataset had additional data in the sales table that was not present in the other tables, I chose to omit this data from my database)



(I used the SQL Developer Import function since my database contained tables with 100,000+ rows)



(Import via .csv) <--Use the calendar_import.csv and sales_import.csv instead of originals

PL/SQL Blocks

1. Retrieve the total revenue, total cost, total profit, and product name associated with the order ID ORD00000179.

INPUT:

DECLARE

```
v_product_name product.product_name%type;
v_revenue      sales.revenue%type;
v_cost         sales.cost%type;
v_profit       sales.profit%type;
v_order_id     sales.order_id%type := 'ORD00000179';
```

BEGIN

```
select p.product_name, s.revenue, s.cost, s.profit
into v_product_name, v_revenue, v_cost, v_profit
from sales s, product p
where s.product_id = p.product_id
and s.order_id = v_order_id;
```

```
DBMS_OUTPUT.PUT_LINE('Order ID: ' || v_order_id);
```

```
DBMS_OUTPUT.PUT_LINE(RPAD('-', 100, '-')); --Use rpad to add "-" to right side
```

```
DBMS_OUTPUT.PUT_LINE('Product Name: ' || v_product_name);
```

```
--Use TO_CHAR for money format
```

```
DBMS_OUTPUT.PUT_LINE('Revenue: ' || TO_CHAR(v_revenue, '$999,999,999.00'));
```

```
DBMS_OUTPUT.PUT_LINE('Cost: ' || TO_CHAR(v_cost, '$999,999,999.00'));
```

```
DBMS_OUTPUT.PUT_LINE('Profit: ' || TO_CHAR(v_profit, '$999,999,999.00'));
```

```
DBMS_OUTPUT.PUT_LINE(RPAD('-', 100, '-'));
```

```
END;
```

OUTPUT:

```
Order ID: 0RD00000179
-----
Product Name: Dark Chocolate 70%
Revenue:      $23.27
Cost:         $15.56
Profit:       $7.70
-----

PL/SQL procedure successfully completed.
```

2. A sales event is going on and there will be an additional discount added on top of all orders (including existing discounts). Calculate the discount amount for the different orders based on their total revenue. Orders with above \$50 revenue receive a 10 percent discount, orders between \$25 and \$50 receive a 5 percent discount, and all others receive no discount.

INPUT:

```
/*
```

```
Above 50 dollars - '0RD00000001'
```

```
In between 25 and 50 dollars - '0RD00000004'
```

```
Less than 24 dollars - '0RD00001136'
```

```
*/
```

```
DECLARE
```

```
  v_order_id sales.order_id%type := '0RD00000001';
```

```
  v_revenue number(10,2);
```

```
  --Initialize Variables
```

```
  v_disc_amt number(10,2);
```

```
  v_final_rev number(10,2);
```


BEGIN

```
select SUM(revenue)
into v_revenue
from sales
where order_id = v_order_id;
```

CASE

```
when v_revenue > 50 then
    v_disc_amt := v_revenue * 0.10;
when v_revenue between 25 and 50 then
    v_disc_amt := v_revenue * 0.05;
else
    v_disc_amt := 0;
```

END CASE;

--Calculate Final Revenue

```
v_final_rev := v_revenue - v_disc_amt;
```

```
DBMS_OUTPUT.PUT_LINE('Order ID: ' || v_order_id);
```

```
DBMS_OUTPUT.PUT_LINE(RPAD('-', 100, '-'));
```

```
DBMS_OUTPUT.PUT_LINE('Original Revenue: ' || TO_CHAR(v_revenue, '$999,999.00'));
```

```
DBMS_OUTPUT.PUT_LINE('Discount Applied: ' || TO_CHAR(v_disc_amt, '$999,999.00'));
```

```
DBMS_OUTPUT.PUT_LINE('Final Revenue: ' || TO_CHAR(v_final_rev, '$999,999.00'));
```

```
DBMS_OUTPUT.PUT_LINE(RPAD('-', 100, '-'));
```

END;

OUTPUT:

```
Order ID: 0RD00000001
-----
Original Revenue:      $61.33
Discount Applied:      $6.13
Final Revenue:         $55.20
-----

PL/SQL procedure successfully completed.
```

3. Loop through all sales records and display the order date, customer ID, quantity, and product name for orders where the quantity is greater than 4 and the product is Milk Chocolate 60%.

INPUT:

DECLARE

CURSOR sales_cursor IS --Cursor b/c returning several rows

select c.date_value,

s.customer_id,

s.quantity,

p.product_name

from sales s, calendar c, product p

where s.product_id = p.product_id

and s.date_id = c.date_id

and p.product_name = 'Milk Chocolate 60%'

and s.quantity > 4;

BEGIN

DBMS_OUTPUT.PUT_LINE(RPAD('-', 80, '-'));

```

DBMS_OUTPUT.PUT_LINE('High Quantity Orders (Quantity > 4)');

DBMS_OUTPUT.PUT_LINE(RPAD('-', 80, '-'));

for rec in sales_cursor LOOP

    DBMS_OUTPUT.PUT_LINE(

        RPAD('Date: ' || rec.date_value, 20) ||

        RPAD('Customer: ' || rec.customer_id, 20) ||

        RPAD('Qty: ' || rec.quantity, 10) || --Quantity was having issues displaying so I used
TO__CHAR

        'Product: ' || rec.product_name

    );

END LOOP;

DBMS_OUTPUT.PUT_LINE(RPAD('-', 80, '-'));

END;

```

OUTPUT:

```

-----
High Quantity Orders (Quantity > 4)
-----
Date: 16-MAY-24      Customer: C013007  Qty: 5   Product: Milk Chocolate 60%
Date: 15-APR-23      Customer: C033257  Qty: 5   Product: Milk Chocolate 60%
Date: 13-OCT-23      Customer: C019996  Qty: 5   Product: Milk Chocolate 60%
Date: 16-NOV-23      Customer: C001580  Qty: 5   Product: Milk Chocolate 60%
Date: 16-MAY-23      Customer: C041527  Qty: 5   Product: Milk Chocolate 60%
Date: 08-JUL-24      Customer: C037099  Qty: 5   Product: Milk Chocolate 60%
Date: 07-MAR-23      Customer: C013885  Qty: 5   Product: Milk Chocolate 60%
Date: 14-JUN-23      Customer: C008445  Qty: 5   Product: Milk Chocolate 60%
Date: 30-DEC-24      Customer: C036610  Qty: 5   Product: Milk Chocolate 60%
Date: 24-SEP-23      Customer: C030673  Qty: 5   Product: Milk Chocolate 60%
Date: 07-FEB-24      Customer: C031306  Qty: 5   Product: Milk Chocolate 60%
Date: 10-DEC-23      Customer: C018355  Qty: 5   Product: Milk Chocolate 60%
Date: 19-OCT-24      Customer: C013642  Qty: 5   Product: Milk Chocolate 60%
Date: 05-JAN-23      Customer: C019859  Qty: 5   Product: Milk Chocolate 60%
Date: 26-JAN-24      Customer: C007278  Qty: 5   Product: Milk Chocolate 60%
Date: 27-FEB-24      Customer: C036749  Qty: 5   Product: Milk Chocolate 60%
Date: 04-DEC-24      Customer: C001007  Qty: 5   Product: Milk Chocolate 60%
Date: 01-JUL-24      Customer: C022975  Qty: 5   Product: Milk Chocolate 60%
Date: 28-OCT-23      Customer: C044389  Qty: 5   Product: Milk Chocolate 60%
Date: 23-OCT-23      Customer: C048647  Qty: 5   Product: Milk Chocolate 60%
Date: 27-OCT-23      Customer: C019587  Qty: 5   Product: Milk Chocolate 60%
Date: 10-JAN-24      Customer: C012129  Qty: 5   Product: Milk Chocolate 60%
...

PL/SQL procedure successfully completed.

```

4. Select all sales records from a specific date and classify each sale based on its revenue into three categories:

- High if revenue is greater than or equal to \$50
- Medium if revenue is between \$25 and \$50
- Low if revenue is less than \$25

INPUT:

DECLARE

cursor sales_cursor is

select s.order_id,

c.date_value,

s.revenue

from sales s, calendar c

where s.date_id = c.date_id

and s.quantity > 4

and s.date_id= 'C572';

--Initialize Classification Variable

v_classification varchar(20);

BEGIN

DBMS_OUTPUT.PUT_LINE(RPAD('-', 100, '-'));

DBMS_OUTPUT.PUT_LINE('Order | Date | Revenue | Classification');

DBMS_OUTPUT.PUT_LINE(RPAD('-', 100, '-'));

for rec in sales_cursor LOOP

--Revenue Classification

CASE

when rec.revenue > 50 THEN

v_classification := 'High';

when rec.revenue between 25 and 50 then

v_classification := 'Medium';

else

v_classification := 'Low';

END CASE;

DBMS_OUTPUT.PUT_LINE(

RPAD('Order: ' || rec.order_id, 20) ||

RPAD('Date: ' || TO_CHAR(rec.date_value, 'MM/DD/YYYY'), 20) ||

RPAD('Rev: ' || TO_CHAR(rec.revenue, '\$999,999.00'), 20) ||

'Class: ' || v_classification

);

END LOOP;

DBMS_OUTPUT.PUT_LINE(RPAD('-', 100, '-'));

END;

OUTPUT:

Order	Date	Revenue	Classification	
Order: 0RD00001861	Date: 07/25/2024	Rev:	\$59.10	Class: High
Order: 0RD00003009	Date: 07/25/2024	Rev:	\$71.10	Class: High
Order: 0RD00007745	Date: 07/25/2024	Rev:	\$24.35	Class: Low
Order: 0RD00027646	Date: 07/25/2024	Rev:	\$18.45	Class: Low
Order: 0RD00014365	Date: 07/25/2024	Rev:	\$16.25	Class: Low
Order: 0RD00039485	Date: 07/25/2024	Rev:	\$67.15	Class: High
Order: 0RD00036706	Date: 07/25/2024	Rev:	\$53.89	Class: High
Order: 0RD00042540	Date: 07/25/2024	Rev:	\$60.95	Class: High
Order: 0RD00032370	Date: 07/25/2024	Rev:	\$62.51	Class: High
Order: 0RD00034656	Date: 07/25/2024	Rev:	\$29.44	Class: Medium
Order: 0RD00029170	Date: 07/25/2024	Rev:	\$52.36	Class: High
Order: 0RD00030951	Date: 07/25/2024	Rev:	\$30.00	Class: Medium
Order: 0RD00054649	Date: 07/25/2024	Rev:	\$29.93	Class: Medium
Order: 0RD00053356	Date: 07/25/2024	Rev:	\$45.85	Class: Medium
Order: 0RD00055089	Date: 07/25/2024	Rev:	\$35.35	Class: Medium
Order: 0RD00051860	Date: 07/25/2024	Rev:	\$18.35	Class: Low
Order: 0RD00050635	Date: 07/25/2024	Rev:	\$72.55	Class: High
Order: 0RD00050706	Date: 07/25/2024	Rev:	\$50.85	Class: High
Order: 0RD00049010	Date: 07/25/2024	Rev:	\$35.60	Class: Medium
Order: 0RD00049421	Date: 07/25/2024	Rev:	\$41.40	Class: Medium
Order: 0RD00047778	Date: 07/25/2024	Rev:	\$43.50	Class: Medium
Order: 0RD00048374	Date: 07/25/2024	Rev:	\$58.35	Class: High
...				

PL/SQL procedure successfully completed.

Output is truncated. View as a [scrollable element](#) or open in a [text editor](#). Adjust cell output [settings](#)...

5. Count the number of purchases made by the top 50 customers with the highest number of purchases.

INPUT:

DECLARE

cursor cust_cursor is

select *

from (

select customer_id,

count(*) total_purchases

from sales

group by CUSTOMER_ID

order by total_purchases desc)

where rownum <= 50; --First 50 rows

BEGIN

DBMS_OUTPUT.PUT_LINE(RPAD('-', 60, '-'));

DBMS_OUTPUT.PUT_LINE('Customer Purchase Summary');

DBMS_OUTPUT.PUT_LINE(RPAD('-', 60, '-'));

for rec in cust_cursor LOOP

DBMS_OUTPUT.PUT_LINE(

RPAD('Customer: ' || rec.customer_id, 25) ||

'Purchases: ' || rec.total_purchases);

END LOOP;

DBMS_OUTPUT.PUT_LINE(RPAD('-', 60, '-'));

END;

OUTPUT:

```
-----
Customer Purchase Summary
-----
Customer: C044952      Purchases: 40
Customer: C033309      Purchases: 39
Customer: C005182      Purchases: 39
Customer: C038733      Purchases: 38
Customer: C046258      Purchases: 38
Customer: C023152      Purchases: 38
Customer: C017174      Purchases: 38
Customer: C031818      Purchases: 37
Customer: C007731      Purchases: 37
Customer: C017393      Purchases: 37
Customer: C029306      Purchases: 37
Customer: C026507      Purchases: 37
Customer: C023828      Purchases: 37
Customer: C038641      Purchases: 37
Customer: C001981      Purchases: 37
Customer: C006389      Purchases: 37
Customer: C047335      Purchases: 36
Customer: C049199      Purchases: 36
Customer: C008999      Purchases: 36
Customer: C004918      Purchases: 36
Customer: C020020      Purchases: 36
Customer: C034132      Purchases: 36
...

PL/SQL procedure successfully completed.

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

6. Try retrieving a customer that doesn't exist from sales, then link the output to the oracle error.

INPUT:

DECLARE

ex_cust_id EXCEPTION;

pragma exception_init(ex_cust_id, -2291);

BEGIN

update sales

set customer_id = 'C050001' --Customer table ends at C050000

WHERE order_id = 'ORD00000001';

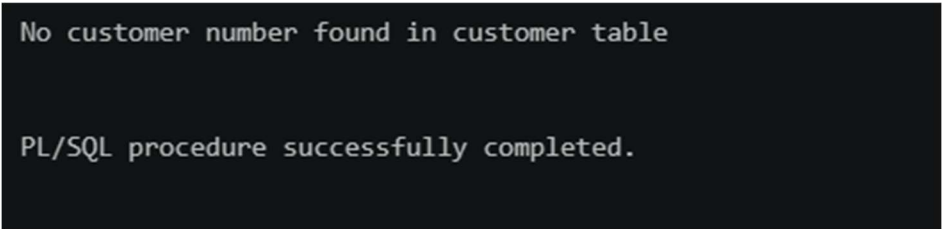
EXCEPTION

when ex_cust_id then

DBMS_OUTPUT.PUT_LINE('No customer number found in customer table');

END;

OUTPUT:



```
No customer number found in customer table

PL/SQL procedure successfully completed.
```

7. Attempt to insert a new customer into the customer table and display an error message if the customer ID already exists.

INPUT:

DECLARE

ex_customer_pk EXCEPTION;

pragma exception_init(ex_customer_pk, -00001);

BEGIN

insert into customer

values ('C000001', 25, 'Male', 1, TO_DATE('05/21/2025','MM/DD/YYYY'));

EXCEPTION

when ex_customer_pk then

DBMS_OUTPUT.PUT_LINE('Primary key violation - Customer ID already exists');

END;

OUTPUT:

```
Primary key violation - Customer ID already exists

PL/SQL procedure successfully completed.
```

8. Create a function that returns the total revenue generated by a given product.

--Drop Function

drop function get_total_revenue;

INPUT:

--Function

CREATE or replace function get_total_revenue (

 p_product_id in varchar2

)

return number

is

 v_total_revenue Number(12,2);

BEGIN

 select sum(revenue)

 into v_total_revenue

 from sales

 where product_id = p_product_id;

 return v_total_revenue;

END;

OUTPUT:

```
Function GET_TOTAL_REVENUE compiled
```

INPUT:

DECLARE

 v_result number;

BEGIN

 v_result := get_total_revenue('P0160');

 DBMS_OUTPUT.PUT_LINE('Total Revenue: ' || TO_CHAR(v_result, '\$999,999.00'));

END;

OUTPUT:

```
Total Revenue:  $128,062.45
```

```
PL/SQL procedure successfully completed.
```

9. Create a procedure to reset all discounts in the sales table to zero.

INPUT:

--Drop Procedure

drop procedure reset_all_discounts

--Drop copy of sales table

drop table sales_2;

--Create copy of sales table

create table sales_2 as select * from SALES;

OUTPUT:

```
Table SALES_2 created.
```

INPUT:

--Create Procedure

CREATE or replace procedure reset_all_discounts

IS

BEGIN

 update SALES_2

 set discount = 0;

DBMS_OUTPUT.PUT_LINE('All discounts have been reset to zero.');

END;

OUTPUT:

```
Procedure RESET_ALL_DISCOUNTS compiled
```

INPUT:

```
--Run Procedure  
  
BEGIN  
  
    reset_all_discounts;  
  
END;
```

OUTPUT:

```
All discounts have been reset to zero.  
  
PL/SQL procedure successfully completed.
```

INPUT:

```
select * from sales_2
```

OUTPUT:

ORDER_ID	CUSTOMER_ID	QUANTITY	UNIT_PRICE	DISCOUNT	REVENUE	COST	PROFIT
1075	C030275	2	3.46	0	6.92	4.05	2.87
1051	C019226	3	8.87	0	22.62	11.71	10.91
1059	C000680	3	9.81	0	23.54	13.71	9.83
1032	C021105	1	4.89	0	4.89	3.09	1.8
1018	C001905	3	5.43	0	16.29	10.12	6.17
1013	C002086	4	5.08	0	20.32	12.08	8.24
1033	C005750	2	5.04	0	8.06	4.67	3.39
1058	C024357	3	9.67	0	26.11	15.29	10.82
1036	C043359	3	5.58	0	16.74	9.03	7.71
1014	C014310	2	8.72	0	17.44	9.44	8
1011	C026414	4	10.39	0	41.56	28.55	13.01
1055	C013007	5	9.03	0	36.12	20.54	15.58
1017	C041501	1	7.71	0	6.17	3.82	2.34
1038	C007934	3	7.83	0	23.49	11.88	11.61

(All discounts were reset to zero)

10. Use a package to perform multiple operations including; calculating total revenue for a product, retrieving product information and applying discounts, then displaying the results.

INPUT:

--Drop Package

drop PACKAGE sales_pkg;

--Drop copy of sales table

drop table sales_2;

--Create copy of sales table

create table sales_2 as select * from SALES;

OUTPUT:

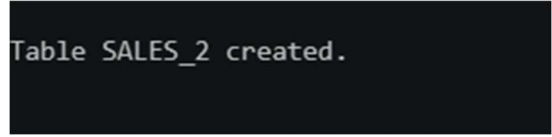


Table SALES_2 created.

INPUT:

--Create Package

CREATE or replace package sales_pkg as

function get_total_revenue (

p_product_id in varchar2

) return number;

function get_product_name (

p_product_id in varchar2

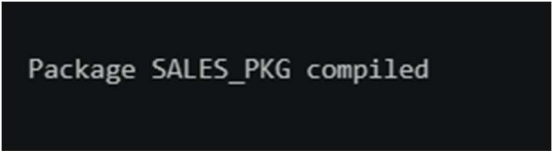
) return varchar2;

```
procedure apply_discount (  
    p_order_id in varchar2,  
    p_discount in number  
);
```

```
procedure reset_discount (  
    p_order_id in varchar2,  
    p_discount in number  
);
```

```
END sales_pkg;
```

OUTPUT:



```
Package SALES_PKG compiled
```

INPUT:

```
--Package Body
```

```
CREATE or replace package body sales_pkg as
```

```
--Function: Total Revenue
```

```
function get_total_revenue (  
    p_product_id in varchar2  
) return NUMBER
```

```
is
```

```

    v_total number(12,2);

BEGIN

    select SUM(revenue)

    into v_total

    from sales_2

    where product_id = p_product_id;


    return v_total;

END;

--Function: Product Name

Function get_product_name (

    p_product_id in varchar2

) return varchar2

is

    v_name product.product_name%type;

BEGIN

    select product_name

    into v_name

    from product

    where product_id = p_product_id;


    return v_name;

END;


--Procedure: Apply Discount

procedure apply_discount (

    p_order_id in varchar2,

```



```
    p_discount in number
)
is
BEGIN
    Update sales_2
    set revenue = revenue * (1 - p_discount)
    where order_id = p_order_id;
END;
```

--Procedure: Reset Discount

```
procedure reset_discount (
    p_order_id in varchar2,
    p_discount in number
)
is
BEGIN
    update sales_2
    set revenue = revenue / (1 - p_discount)
    where order_id = p_order_id;
END;
```

```
END sales_pkg;
```

OUTPUT:

```
Package Body SALES_PKG compiled
```

INPUT:

```
--Main Program
```

```
DECLARE
```

```
    v_revenue NUMBER;
```

```
    v_name    VARCHAR2(100);
```

```
    v_old_rev NUMBER;
```

```
    v_new_rev NUMBER;
```

```
BEGIN
```

```
    --Get total revenue
```

```
    v_revenue := sales_pkg.get_total_revenue('P0001');
```

```
    DBMS_OUTPUT.PUT_LINE('Total Revenue: ' || TO_CHAR(v_revenue, '$999,999.00'));
```

```
    --Get product name
```

```
    v_name := sales_pkg.get_product_name('P0001');
```

```
    DBMS_OUTPUT.PUT_LINE('Product Name: ' || v_name);
```

```
    --Get original revenue for an order
```

```
    SELECT revenue INTO v_old_rev
```

```
    FROM sales_2
```

```
    WHERE order_id = 'ORD00000001';
```

```
    --Print original revenue
```

```
DBMS_OUTPUT.PUT_LINE('Original Revenue: ' || v_old_rev);
```

```
--Apply discount
```

```
sales_pkg.apply_discount('ORD00000001', 0.10);
```

```
select revenue into v_new_rev
```

```
from sales_2
```

```
where order_id = 'ORD00000001';
```

```
DBMS_OUTPUT.PUT_LINE('After Discount: ' || v_new_rev);
```

```
--Reset discount (Restores revenue to original value)
```

```
sales_pkg.reset_discount('ORD00000001', 0.10);
```

```
select revenue into v_old_rev
```

```
from sales_2
```

```
where order_id = 'ORD00000001';
```

```
DBMS_OUTPUT.PUT_LINE('Reset Revenue: ' || v_old_rev);
```

```
END;
```

OUTPUT:

```
Total Revenue: $128,281.48  
Product Name: White Chocolate 80%  
Original Revenue: 61.33  
After Discount: 55.2  
Reset Revenue: 61.33
```

```
PL/SQL procedure successfully completed.
```
